



Univerzitet u Nišu
Elektronski Fakultet



Optimizacija upita u PostgreSQL

Mentor:
doc. dr Aleksandar Stanimirović

Student:
Filip Stamenković 2027

Niš, 2026.

Sadržaj

1. Uvod	3
2. Obrada i optimizacija upita u PostgreSQL-u	4
2.1 Proces obrade SQL upita	4
2.2 PostgreSQL planer i optimizator	5
2.3 Cost-based optimizacija	6
2.4 Statistike, selektivnost i procena kardinalnosti	7
3. Planovi izvršavanja upita	7
3.1 EXPLAIN i EXPLAIN ANALYZE	8
3.2 Sequential Scan, Index Scan i Index Only Scan	9
3.3 Nested Loop, Hash Join i Merge Join	10
3.4 Troškovi i procena broja redova	11
4. Tehnike optimizacije upita	12
4.1 Optimizacija korišćenjem indeksa	13
4.2 Composite indeksi	13
4.3 Optimizacija JOIN operacija	14
4.4 Query rewriting	15
4.5 Statistike i ANALYZE	16
5. Praktična demonstracija optimizacije upita	17
5.1 Testno okruženje i podaci	17
5.2 Eksperiment 1 – Uticaj B-tree indeksa na filtriranje podataka	18
5.3 Eksperiment 2 – Jednokolonski i višekolonski indeks	21
5.4 Eksperiment 3 – Uticaj načina pisanja upita na korišćenje indeksa	24
5.5 Eksperiment 4 – Optimizacija JOIN operacije	26
5.6 Eksperiment 5 – Selektivnost i izbor plana izvršavanja	29
6. Zaključak	32
7. Literatura	33

1. Uvod

Savremeni informacioni sistemi često rade sa velikim količinama podataka, pri čemu se od sistema za upravljanje bazama podataka očekuje da omoguće efikasno čuvanje, pretraživanje i obradu tih podataka. Sa povećanjem količine podataka i složenosti aplikacija, vreme potrebno za izvršavanje SQL upita može značajno uticati na ukupne performanse sistema. Zbog toga optimizacija upita predstavlja jedan od važnih aspekata projektovanja i održavanja sistema baza podataka.

Prilikom izvršavanja SQL upita korisnik definiše podatke koje želi da dobije, ali ne određuje način na koji će sistem do tih podataka doći. Sistem za upravljanje bazom podataka mora da odredi odgovarajući način pristupa tabelama, redosled izvršavanja operacija, način povezivanja tabela i mogućnost korišćenja postojećih indeksa. Za jedan SQL upit može postojati veliki broj različitih planova izvršavanja koji daju isti rezultat, ali se mogu značajno razlikovati prema potrebnom vremenu izvršavanja i korišćenju sistemskih resursa. Zadatak optimizatora upita je da među mogućim planovima pronađe plan čiji je procenjeni trošak izvršavanja najpovoljniji.

PostgreSQL koristi optimizator zasnovan na proceni troškova (cost-based optimizer). Prilikom izbora plana izvršavanja uzimaju se u obzir različiti faktori, kao što su statistički podaci o tabelama i kolonama, procenjeni broj redova koji zadovoljava određene uslove, dostupni indeksi i procenjeni troškovi procesorskih i ulazno-izlaznih operacija. Na osnovu ovih informacija PostgreSQL može, na primer, da odluči da li će podatke pročitati sekvencijalnim prolaskom kroz tabelu ili korišćenjem indeksa, kao i koji će algoritam koristiti prilikom izvršavanja operacija spajanja tabela.

Važan alat za analizu ovih odluka predstavljaju PostgreSQL komande EXPLAIN i EXPLAIN ANALYZE. One omogućavaju prikaz plana izvršavanja upita, procenjenih troškova pojedinačnih operacija i, u slučaju komande EXPLAIN ANALYZE, stvarnih rezultata izvršavanja. Analizom ovih podataka moguće je identifikovati delove upita koji predstavljaju usko grlo i utvrditi da li primena indeksa, izmena samog upita ili ažuriranje statističkih podataka može dovesti do boljih performansi.

2. Obrada i optimizacija upita u PostgreSQL-u

Kada korisnik pošalje SQL upit PostgreSQL sistemu, upit se ne izvršava direktno u obliku u kojem je napisan. Pre samog pristupa podacima prolazi kroz više faza obrade čiji je cilj da se proverí njegova ispravnost, odredi način izvršavanja i izabere plan koji bi trebalo da omogući efikasno dobijanje rezultata. Među najvažnijim fazama nalaze se parsiranje upita, transformacija i prepisivanje upita, planiranje i optimizacija, a zatim izvršavanje izabranog plana [6][7].

2.1 Proces obrade SQL upita

Prva faza obrade je **parsiranje** (*parsing*). PostgreSQL analizira tekst SQL naredbe, proverava njenu sintaksu i formira internu strukturu koja predstavlja napisani upit. Nakon toga vrši se semantička analiza, tokom koje se, između ostalog, identifikuju tabele, kolone, operatori i tipovi podataka na koje se upit odnosi.

Nakon parsiranja sledi faza **prepisivanja upita** (*query rewriting*). PostgreSQL u ovoj fazi može da transformiše početnu internu reprezentaciju upita na osnovu pravila definisanih u sistemu. Ova faza je naročito značajna kod objekata kao što su pogledi (*views*), čije korišćenje može zahtevati proširivanje originalnog upita odgovarajućim definicijama.

Rezultat prethodnih faza prosleđuje se **planeru i optimizatoru**. Njihov zadatak je da razmotre moguće načine izvršavanja upita. Isti rezultat često je moguće dobiti korišćenjem različitih metoda pristupa podacima, različitim redosledom povezivanja tabela i korišćenjem različitih algoritama za izvršavanje pojedinačnih operacija.

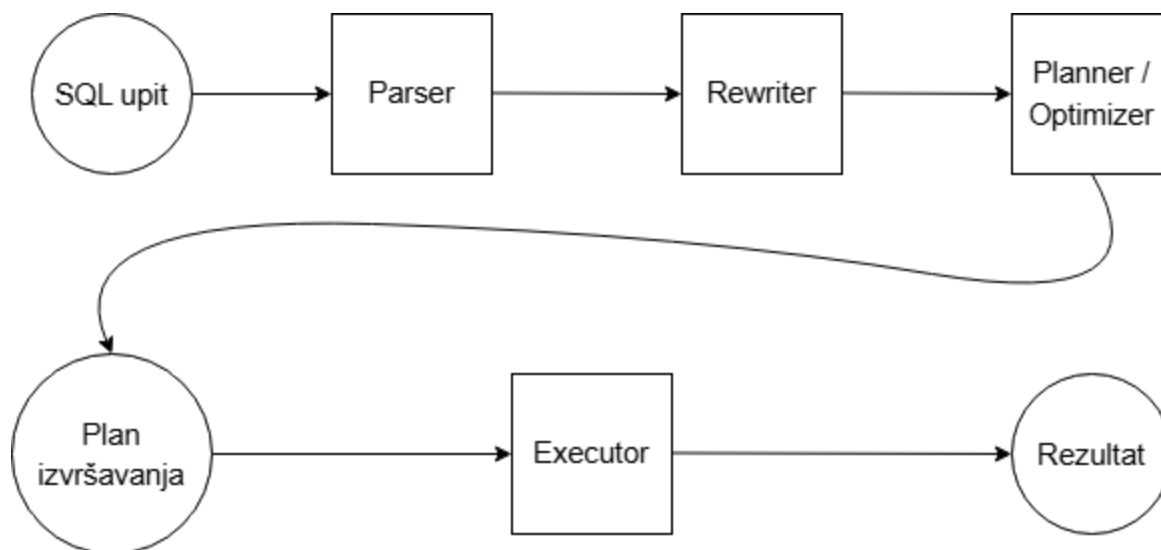
Na primer, za jednostavan upit:

```
SELECT *  
FROM orders  
WHERE customer_id = 100;
```

PostgreSQL može razmatrati sekvencijalno čitanje tabele ili pristup podacima korišćenjem odgovarajućeg indeksa. Oba pristupa daju isti rezultat, ali njihove performanse zavise od veličine tabele, broja redova koji ispunjavaju uslov, raspoloživih indeksa i drugih karakteristika sistema.

Nakon izbora plana, faza **izvršavanja** (*execution*) koristi odabrani plan za pristup podacima i generisanje rezultata. Plan se sastoji od operatora koji predstavljaju konkretne operacije, kao što su čitanje tabele, filtriranje, sortiranje, agregacija ili spajanje podataka iz više tabela.

Proces se pojednostavljeno može predstaviti kao na slici 1:



Slika 1: Faze obrade i proces u izvršavanju sql upita.

Ovakva podela omogućava da korisnik SQL-om definiše koje podatke želi, dok PostgreSQL određuje na koji način će ti podaci biti pronađeni i obrađeni.

2.2 PostgreSQL planer i optimizator

Planer predstavlja jednu od ključnih komponenti PostgreSQL-a sa aspekta performansi upita. Za dobijeni upit on razmatra različite moguće planove izvršavanja i pokušava da izabere onaj sa najmanjim procenjenim troškom [1].

Kod pristupa jednoj tabeli planer može, na primer, razmatrati *Sequential Scan* (Sekvencijalno skeniranje - čitanje cele tabele od početka do kraja), *Index Scan* (Skeniranje indeksa - direktno pronalaženje tačnih redova pomoću indeksa), ili *Bitmap Heap Scan* (Skeniranje pomoću bitmape - kombinacija predhodna dva pristupa radi uštede vremena) [1]. Kod upita koji povezuju više tabela broj mogućnosti postaje znatno veći, jer je potrebno odabrati i redosled povezivanja tabela, kao i odgovarajući JOIN algoritam.

Posmatrajmo upit:

```

SELECT c.name, o.id, o.total
FROM customers c
JOIN orders o ON c.id = o.customer_id
WHERE c.id = 100;

```

PostgreSQL mora da donese više odluka. Potrebno je odrediti kako pronaći odgovarajućeg korisnika, kako pronaći njegove porudžbine i koji način spajanja podataka koristiti. U zavisnosti od podataka i dostupnih indeksa, mogu se dobiti različiti planovi izvršavanja.

Značajno je da planer ne bira plan samo na osnovu postojanja indeksa. Ukoliko uslov obuhvata veliki deo tabele, sekvencijalno čitanje može biti jeftinije od velikog broja pojedinačnih pristupa podacima preko indeksa. Zbog toga postojanje indeksa ne znači da će ga PostgreSQL nužno koristiti.

Sa povećanjem broja tabela u upitu raste i broj mogućih kombinacija njihovog povezivanja. Planer zbog toga mora da pronađe dovoljno dobar plan bez ispitivanja svake teoretski moguće kombinacije kod veoma složenih upita. Optimizacija tako predstavlja kompromis između vremena utrošenog na pronalaženje plana i očekivanog vremena njegovog izvršavanja [6].

2.3 Cost-based optimizacija

PostgreSQL koristi pristup zasnovan na proceni troškova, odnosno *cost-based optimization*. Svakom potencijalnom planu dodeljuje se procenjeni trošak, a planer u opštem slučaju bira plan sa najmanjom procenjenom ukupnom cenom.

Trošak prikazan u PostgreSQL planu nije direktno izražen u milisekundama. On predstavlja internu relativnu vrednost koja omogućava poređenje različitih planova. Na procenu utiču očekivane disk operacije, obrada redova procesorom, izvršavanje operatora i funkcija, kao i drugi faktori [1].

Kod EXPLAIN izlaza mogu se videti vrednosti poput:

```
Seq Scan on orders  
(cost=0.00..18500.00 rows=1000 width=32)
```

Prva vrednost predstavlja *startup cost*, odnosno procenjeni trošak pre nego što operator može da proizvede prvi red rezultata. Druga vrednost predstavlja *total cost*, odnosno procenjeni trošak potreban da operator proizvede sve očekivane redov, *rows* predstavlja procenjeni broj redova, dok *width* predstavlja procenjenju prosečnu veličinu jednog reda izraženu u bajtovima [1].

Ovakav model omogućava poređenje različitih strategija. Na primer, *Index Scan* može imati veoma nizak trošak kada je potrebno pronaći mali broj redova, dok *Sequential Scan* može biti povoljniji kada je potrebno pročitati veliki deo tabele.

PostgreSQL koristi niz konfiguracionih parametara za modelovanje ovih troškova, među kojima su *seq_page_cost*, *random_page_cost*, *cpu_tuple_cost* i drugi. Njihove vrednosti omogućavaju planeru da proceni relativnu cenu pristupa podacima i njihove obrade [4].

Važno je naglasiti da optimizir bira plan na osnovu **procene**, a ne prethodnog izvršavanja konkretnog upita. Zbog toga kvalitet izabranog plana u velikoj meri zavisi od tačnosti informacija kojima planer raspolaže.

2.4 Statistike, selektivnost i procena kardinalnosti

Da bi planer mogao da proceni troškove različitih planova, potrebno je da ima informacije o podacima koji se nalaze u tabelama. PostgreSQL zbog toga održava statističke podatke koji opisuju sadržaj tabela i distribuciju vrednosti pojedinih kolona [2].

Jedan od najvažnijih problema je procena *kardinalnosti*, odnosno procena broja redova koji će nastaviti u sledeći deo plana nakon određene operacije. Sa tim je povezana *selektivnost* uslova, koja opisuje koliki deo redova zadovoljava određeni kriterijum.

Na primer, ako tabela *orders* sadrži milion redova, a planer proceni da uslov:

```
WHERE customer_id = 100
```

vraća samo deset redova, korišćenje indeksa može biti veoma povoljno. Međutim, ako uslov vraća nekoliko stotina hiljada redova, sekvencijalno čitanje tabele može predstavljati efikasnije rešenje.

PostgreSQL prikuplja statistike pomoću komande ANALYZE, koja se takođe automatski izvršava kroz mehanizme održavanja baze [5]. Statistike mogu sadržati informacije kao što su procenjeni broj različitih vrednosti, najčešće vrednosti i informacije o distribuciji vrednosti u koloni [2].

Ako statistike ne odgovaraju trenutnom stanju podataka, procena kardinalnosti može biti netačna. Planer tada može, na primer, očekivati deset redova, dok se tokom stvarnog izvršavanja dobije nekoliko stotina hiljada. Takva greška može dovesti do izbora plana koji je prema proceni bio povoljan, ali se u praksi pokazuje kao neefikasan.

Upravo zbog toga su statistike, selektivnost i procena kardinalnosti usko povezane sa optimizacijom upita. PostgreSQL ne pokušava jednostavno da pronađe indeks ili najkraći plan, već da na osnovu dostupnih informacija proceni cenu različitih mogućnosti i odabere onu za koju očekuje da će zahtevati najmanje resursa.

3. Planovi izvršavanja upita

Plan izvršavanja predstavlja način na koji PostgreSQL namerava da izvrši određeni SQL upit. Nakon analize mogućih strategija, planer bira plan koji prema njegovoj proceni ima najniži

trošak. Plan se sastoji od niza operatora koji opisuju operacije pristupa podacima, filtriranja, spajanja, sortiranja i agregacije.

Razumevanje plana izvršavanja predstavlja jedan od osnovnih koraka pri optimizaciji upita. Umesto pretpostavke da je određen upit spor zbog njegove sintakse ili veličine tabele, analizom plana moguće je utvrditi koje operacije PostgreSQL stvarno izvršava i gde se nalazi potencijalni problem.

3.1 EXPLAIN i EXPLAIN ANALYZE

PostgreSQL omogućava pregled plana izvršavanja pomoću komande EXPLAIN. Ona prikazuje plan koji bi PostgreSQL koristio za izvršavanje upita, ali sam upit ne izvršava [1].

Na primer:

```
EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 100;
```

Rezultat može izgledati ovako:

```
Seq Scan on orders
  (cost=0.00..18870.00 rows=10 width=32)
  Filter: (customer_id = 100)
```

U ovom slučaju PostgreSQL je odlučio da izvrši *Sequential Scan*, odnosno da sekvencijalno prođe kroz tabelu *orders* i proveri uslov za pronađene redove.

Pored vrste operatora, EXPLAIN prikazuje nekoliko značajnih procenjenih vrednosti: *cost* predstavlja procenjeni trošak izvršavanja operatora, *rows* očekivani broj redova koje će operator proizvesti, dok *width* predstavlja procenjenju prosečnu veličinu jednog reda u bajtovima.

Za detaljniju analizu koristi se EXPLAIN ANALYZE:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE customer_id = 100;
```

Za razliku od običnog EXPLAIN, ova naredba zaista izvršava upit i prikazuje stvarne podatke o izvršavanju [1]. Rezultat, pored procena planera, sadrži informacije kao što su stvarno vreme izvršavanja i stvarni broj obrađenih redova.

Primer pojednostavljenog rezultata može biti:

```
Seq Scan on orders
  (cost=0.00..18870.00 rows=10 width=32)
  (actual time=0.125..48.321 rows=12 loops=1)
    Filter: (customer_id = 100)
Planning Time: 0.180 ms
Execution Time: 48.370 ms
```

Posebno je korisno poređenje vrednosti *rows* iz procene sa stvarnom vrednošću *rows* dobijenom izvršavanjem. Velika razlika između njih može ukazivati na problem sa procenom kardinalnosti, što može dovesti do izbora neodgovarajućeg plana.

Potrebno je imati u vidu da EXPLAIN ANALYZE stvarno izvršava SQL naredbu. Kod naredbi koje menjaju podatke, kao što su UPDATE ili DELETE, promene će takođe biti izvršene ukoliko se analiza ne obavlja u odgovarajućoj transakciji koja se nakon toga poništava [1].

3.2 Sequential Scan, Index Scan i Index Only Scan

Jedna od najvažnijih odluka planera jeste način pristupa podacima u tabeli. PostgreSQL podržava više metoda, od kojih su za optimizaciju posebno značajni *Sequential Scan*, *Index Scan* i *Index Only Scan*.

Sequential Scan podrazumeva sekvencijalno čitanje stranica tabele i proveru redova u odnosu na zadati uslov. Iako se često smatra sporijom opcijom, sekvencijalno čitanje može biti veoma efikasno kada upit treba da obradi veliki deo tabele.

Na primer:

```
SELECT *
FROM orders
WHERE status = 'COMPLETED';
```

Ako većina porudžbina ima status COMPLETED, korišćenje indeksa ne mora predstavljati najbolji izbor. PostgreSQL može proceniti da je jednostavnije pročitati tabelu sekvencijalno nego izvršavati veliki broj pristupa preko indeksa.

Index Scan koristi indeks kako bi pronašao lokacije redova koji odgovaraju uslovu, nakon čega pristupa odgovarajućim redovima u tabeli. Ovaj pristup je naročito efikasan kada uslov vraća mali deo ukupnog broja redova [3].

Na primer, nakon kreiranja indeksa:

```
CREATE INDEX idx_orders_customer_id  
ON orders(customer_id);
```

upit:

```
SELECT *  
FROM orders  
WHERE customer_id = 100;
```

može koristiti:

```
Index Scan using idx_orders_customer_id on orders
```

Treća mogućnost je **Index Only Scan**. U određenim situacijama svi podaci potrebni za izvršavanje upita mogu se pronaći direktno u indeksu, čime se može izbeći pristup odgovarajućim redovima tabele [3].

Na primer:

```
SELECT customer_id  
FROM orders  
WHERE customer_id = 100;
```

ako postoji odgovarajući indeks nad *customer_id*, PostgreSQL može koristiti *Index Only Scan*.

Ipak, zbog PostgreSQL MVCC mehanizma, postojanje svih potrebnih kolona u indeksu samo po sebi nije uvek dovoljno da se potpuno izbegne pristup tabeli. PostgreSQL koristi *visibility map* kako bi utvrdio da li se vidljivost redova može potvrditi bez dodatnog pristupa heap stranicama [3].

Izbor između ovih metoda zato zavisi od količine podataka, selektivnosti uslova, statistika, strukture indeksa i procenjene cene pristupa podacima.

3.3 Nested Loop, Hash Join i Merge Join

Kod upita koji povezuju podatke iz više tabela, PostgreSQL mora da izabere način na koji će izvršiti JOIN operaciju. Tri osnovna algoritma su *Nested Loop*, *Hash Join* i *Merge Join*.

Nested Loop funkcioniše tako što za redove jednog ulaza traži odgovarajuće redove u drugom. Ovakav pristup može biti veoma efikasan kada jedan od ulaza sadrži mali broj redova, posebno ako nad kolonom za povezivanje druge tabele postoji odgovarajući indeks [6][7].

Na primer:

```
SELECT c.name, o.id
FROM customers c
JOIN orders o ON o.customer_id = c.id
WHERE c.id = 100;
```

Pošto uslov $c.id = 100$ verovatno vraća samo jednog korisnika, PostgreSQL može pronaći tog korisnika i zatim preko indeksa pronaći njegove porudžbine. U takvoj situaciji *Nested Loop* predstavlja prirodan izbor.

Hash Join je pogodan za povezivanje većih skupova podataka na osnovu uslova jednakosti. PostgreSQL nad jednim skupom podataka formira hash strukturu, a zatim redove drugog skupa poredi sa njom [6][7].

Plan može izgledati ovako:

```
Hash Join
  Hash Cond: (o.customer_id = c.id)
    -> Seq Scan on orders o
    -> Hash
        -> Seq Scan on customers c
```

Ovakav algoritam često je efikasan kada je potrebno povezati značajan broj redova i ne postoji mogućnost da se mali broj odgovarajućih redova pronađe efikasnim indeksnim pristupom.

Merge Join koristi uređene ulazne skupove podataka. Pošto su podaci sortirani prema kolonama koje učestvuju u povezivanju, PostgreSQL može paralelno prolaziti kroz oba skupa i pronalaziti odgovarajuće vrednosti [6][7].

Njegova efikasnost zavisi od toga da li su podaci već dostupni u potrebnom redosledu. Ukoliko je pre povezivanja potrebno izvršiti dodatno sortiranje velikih skupova, cena takvog plana može značajno porasti.

Ne postoji jedan JOIN algoritam koji je optimalan u svim situacijama. Izbor zavisi od veličine ulaznih skupova, uslova povezivanja, indeksa, redosleda podataka i procenjene kardinalnosti.

3.4 Troškovi i procena broja redova

Plan izvršavanja u PostgreSQL-u može se posmatrati kao stablo operatora. Operator na dnu stabla najčešće pristupa tabelama ili indeksima, dok operatori iznad njega obrađuju rezultate prethodnih operacija [1].

Na primer:

Hash Join

-> Seq Scan on orders

-> Hash

-> Seq Scan on customers

Izvršavanje se može pratiti od operatora na nižim nivoima prema operatoru na vrhu, koji na kraju proizvodi rezultat upita.

Prilikom analize ovakvog plana nije dovoljno posmatrati samo ukupnu vrednost *cost*. Posebno je važno analizirati procenjeni i stvarni broj redova koji prolaze kroz pojedinačne operatore. Greška u proceni na nižem nivou plana može se preneti na naredne operacije i uticati na izbor JOIN algoritma, načina pristupa podacima ili sortiranja [2].

Na primer, planer može proceniti:

```
rows=100
```

dok EXPLAIN ANALYZE pokaže:

```
actual rows=100000
```

Plan koji je efikasan za obradu stotinu redova ne mora biti efikasan za obradu sto hiljada redova. Zbog toga velika razlika između procenjenog i stvarnog broja redova predstavlja jedan od najvažnijih signala koje treba proveriti prilikom analize sporih upita.

Pored toga, EXPLAIN ANALYZE prikazuje i broj izvršavanja pojedinačnog operatora kroz vrednost *loops* [1]. Kod operatora koji se izvršava veliki broj puta, relativno mala cena jednog izvršavanja može prerasti u značajan deo ukupnog vremena upita.

Plan izvršavanja zato predstavlja osnovni izvor informacija prilikom optimizacije PostgreSQL upita. Analizom načina pristupa podacima, JOIN algoritama, procenjene kardinalnosti i stvarnog vremena izvršavanja moguće je identifikovati uzrok loših performansi.

4. Tehnike optimizacije upita

Nakon analize plana izvršavanja moguće je identifikovati operacije koje predstavljaju potencijalna uska grla i primeniti odgovarajuće tehnike optimizacije. Cilj optimizacije nije samo smanjenje vremena izvršavanja pojedinačnog upita, već i smanjenje količine podataka koje sistem mora da pročita i obradi.

U PostgreSQL-u performanse upita mogu se poboljšati pravilnim korišćenjem indeksa, prilagođavanjem složenih upita, optimizacijom operacija povezivanja tabela i održavanjem statističkih podataka. Izbor odgovarajuće tehnike zavisi od strukture baze, količine i distribucije podataka, kao i vrste upita koji se izvršavaju.

4.1 Optimizacija korišćenjem indeksa

Indeksi predstavljaju jednu od najčešće korišćenih tehnika za poboljšanje performansi upita [3]. Njihova osnovna svrha je omogućavanje pronalaženja određenih redova bez potrebe za sekvencijalnim pregledanjem kompletne tabele.

Posmatrajmo tabelu *orders* sa velikim brojem porudžbina i upit:

```
SELECT *  
FROM orders  
WHERE customer_id = 1500;
```

Ako nad kolonom *customer_id* ne postoji indeks, PostgreSQL može koristiti *Sequential Scan* i proveriti vrednost *customer_id* za veliki broj redova. Kreiranjem indeksa:

```
CREATE INDEX idx_orders_customer_id  
ON orders(customer_id);
```

PostgreSQL dobija mogućnost korišćenja *Index Scan* operacije kako bi direktnije pronašao odgovarajuće redove.

Međutim, kreiranje indeksa ne garantuje da će ga planer koristiti. Ako uslov vraća veliki procenat tabele, sekvencijalno čitanje može imati manji procenjeni trošak. Efikasnost indeksa zato u velikoj meri zavisi od selektivnosti uslova.

Indeksi takođe imaju svoju cenu. Zauzimaju dodatni prostor i moraju biti održavani prilikom INSERT, UPDATE i DELETE operacija [3]. Zbog toga kreiranje velikog broja indeksa bez analize stvarnih upita može negativno uticati na performanse sistema.

4.2 Composite indeksi

Kada se više kolona često zajedno pojavljuje u uslovima upita, moguće je koristiti **višekolonske (composite) indekse**.

Na primer, ako se često izvršava upit:

```
SELECT *  
FROM orders
```

```
WHERE customer_id = 1500
AND created_at >= '2026-01-01';
```

moguće je kreirati indeks:

```
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, created_at);
```

Ovakav indeks može omogućiti efikasnije pronalaženje porudžbina konkretnog korisnika u određenom vremenskom periodu.

Kod višekolonskih indeksa značajan je redosled kolona. B-tree indeks nad (*customer_id*, *created_at*) nije ekvivalentan indeksu nad (*created_at*, *customer_id*). PostgreSQL B-tree višekolonski indeks najefikasnije ograničava deo indeksa koji mora biti pregledan kada postoje uslovi nad vodećim kolonama indeksa [3].

Na primer, indeks:

```
(customer_id, created_at)
```

prirodno odgovara upitima koji filtriraju prema *customer_id*, kao i upitima koji koriste obe kolone. Upit koji filtrira samo prema *created_at* može u nekim okolnostima ipak koristiti takav indeks, ali on često neće biti optimalan izbor za taj obrazac pristupa.

Zbog toga strukturu višekolonskog indeksa treba određivati na osnovu stvarnih upita, njihove selektivnosti i načina na koji se kolone koriste.

4.3 Optimizacija JOIN operacija

Operacije povezivanja tabela mogu predstavljati jedan od najskupljih delova složenih SQL upita. Njihove performanse zavise od količine podataka koja ulazi u JOIN, dostupnih indeksa i algoritma povezivanja koji planer odabere.

Jedna od osnovnih tehnika optimizacije jeste postojanje odgovarajućih indeksa nad kolonama koje se često koriste za povezivanje tabela. Na primer:

```
SELECT c.name, o.id, o.total
FROM customers c
JOIN orders o ON o.customer_id = c.id
WHERE c.id = 1500;
```

Indeks nad *orders.customer_id* omogućava PostgreSQL-u da nakon pronalaženja korisnika efikasno pronađe njegove porudžbine.

Drugi važan faktor je količina podataka koja mora da učestvuje u povezivanju. Kada je moguće rano eliminisati nepotrebne redove selektivnim uslovima, smanjuje se količina podataka koju naredni operatori moraju da obrađuju.

PostgreSQL planer može menjati redosled povezivanja tabela i birati između *Nested Loop*, *Hash Join* i *Merge Join* algoritama [1][6]. Zbog toga ručno menjanje redosleda tabela napisanog u jednostavnom INNER JOIN upitu često neće samo po sebi promeniti redosled izvršavanja. Efikasniji pristup je omogućiti planeru kvalitetne statistike i odgovarajuće indekse kako bi mogao da proceni različite mogućnosti.

4.4 Query rewriting

Način na koji je SQL upit napisan može uticati na mogućnosti njegove optimizacije. **Query rewriting** podrazumeva preoblikovanje upita uz očuvanje njegovog rezultata, sa ciljem omogućavanja efikasnijeg izvršavanja.

Jedan čest problem predstavlja primena funkcije nad indeksiranom kolonom u uslovu. Na primer [3][4]:

```
SELECT *
FROM orders
WHERE DATE(created_at) = DATE '2026-05-10';
```

Ako postoji običan indeks nad *created_at*, ovakav izraz može otežati njegovo efikasno korišćenje, jer se uslov primenjuje na rezultat funkcije DATE(created_at), a ne direktno na vrednost kolone.

Upit se može napisati kao:

```
SELECT *
FROM orders
WHERE created_at >= TIMESTAMP '2026-05-10 00:00:00'
AND created_at < TIMESTAMP '2026-05-11 00:00:00';
```

Ovim oblikom se direktno definiše opseg vrednosti kolone *created_at*, što odgovarajući B-tree indeks može efikasno da podrži.

Sličan princip važi i za nepotrebnu obradu podataka. Na primer, korišćenje:

```
SELECT *
```

nije poželjno kada je aplikaciji potreban samo mali broj kolona. Izbor samo potrebnih kolona može smanjiti količinu podataka koja se prenosi i obrađuje, a u određenim situacijama omogućiti i korišćenje *Index Only Scan* operacije.

Ipak, nije potrebno ručno prepisivati svaki složen SQL izraz. PostgreSQL već poseduje mehanizme za transformaciju i optimizaciju velikog broja konstrukcija. Zbog toga efekat svake izmene treba proveriti korišćenjem EXPLAIN ANALYZE, umesto pretpostavke da će drugačije napisan upit nužno biti brži [1][4].

4.5 Statistike i ANALYZE

Čak i kada baza poseduje odgovarajuće indekse i dobro napisane upite, netačne statistike mogu dovesti do lošeg plana izvršavanja. PostgreSQL koristi statističke informacije kako bi procenio selektivnost uslova i broj redova koji će prolaziti kroz pojedinačne operatore [2].

Statistike se mogu ažurirati naredbom [5]:

```
ANALYZE orders;
```

ili za čitavu bazu [5]:

```
ANALYZE;
```

U normalnom radu PostgreSQL automatski pokreće ANALYZE kroz *autovacuum* mehanizam kada se ispune odgovarajući uslovi. Međutim, nakon velikih promena podataka može postojati period tokom kojeg statistike ne predstavljaju dovoljno dobro trenutno stanje tabele.

Posebno su problematične kolone sa neravnomernom distribucijom podataka. Pretpostavimo da tabela sadrži milion porudžbina, od kojih je velika većina završena, dok veoma mali broj ima status CANCELLED. Za upit:

```
SELECT *  
FROM orders  
WHERE status = 'CANCELLED';
```

precizna informacija o distribuciji vrednosti može biti ključna za odluku da li koristiti indeks ili sekvencijalno čitanje.

Kada podrazumevani nivo statistike nije dovoljan za određenu kolonu, moguće je povećati količinu statističkih informacija koje PostgreSQL prikuplja [2]:

```
ALTER TABLE orders  
ALTER COLUMN status
```



```
SET STATISTICS 500;
```

Nakon toga potrebno je ponovo izvršiti:

```
ANALYZE orders;
```

Veća vrednost može omogućiti preciznije procene, ali istovremeno povećava količinu statističkih podataka i vreme potrebno za njihovo prikupljanje i planiranje.

Optimizacija upita zato ne predstavlja jednu konkretnu intervenciju, već proces analize i merenja. Kreiranje indeksa, promena strukture upita ili ažuriranje statistika treba da budu zasnovani na konkretnom planu izvršavanja i karakteristikama podataka.

5. Praktična demonstracija optimizacije upita

U praktičnom delu rada analiziraju se performanse različitih SQL upita nad testnom PostgreSQL bazom podataka. Cilj eksperimenata je da se na konkretnim primerima prikaže način na koji PostgreSQL bira plan izvršavanja, kao i uticaj različitih tehnika optimizacije na vreme izvršavanja i količinu obrađenih podataka.

Za analizu se koristi naredba EXPLAIN ANALYZE, koja omogućava poređenje procena PostgreSQL planera sa stvarnim rezultatima izvršavanja. Kod svakog eksperimenta prvo se analizira početno stanje, nakon čega se primenjuje određena tehnika optimizacije i ponavlja isti upit. Na ovaj način moguće je direktno porediti planove izvršavanja pre i nakon optimizacije.

5.1 Testno okruženje i podaci

Za izvođenje eksperimenata formirana je testna baza podataka koja predstavlja pojednostavljeni sistem elektronske trgovine. PostgreSQL je pokrenut u Docker kontejneru, čime je omogućeno jednostavno kreiranje i reprodukovanje testnog okruženja.

Baza se sastoji od četiri povezane tabele:

- *customers* – podaci o korisnicima,
- *products* – podaci o proizvodima,
- *orders* – porudžbine korisnika,
- *order_items* – pojedinačne stavke porudžbina.

Kako bi razlike između različitih planova izvršavanja bile dovoljno izražene, generisana je veća količina testnih podataka. Baza sadrži približno:

Tabela 1: Broj redova po tabeli u kreiranoj bazi podataka.

Tabela	Broj redova
customers	100.000
products	20.000
orders	1.000.000
order_items	2.000.000

Podaci su generisani korišćenjem PostgreSQL funkcije *generate_series()*. Kod pojedinih kolona korišćena je namerno neravnomerna distribucija vrednosti kako bi se u kasnijim eksperimentima mogao analizirati uticaj selektivnosti i statistike na odluke optimizatora.

Na primer, vrednosti kolone *status* u tabeli *orders* generisane su približno u sledećem odnosu:

Tabela 2: Odnos vrednosti u koloni status u tabeli orders.

COMPLETED	70%
PROCESSING	20%
CANCELLED	8%
REFUNDED	2%

Nakon generisanja podataka izvršena je naredba:

ANALYZE;

kako bi PostgreSQL prikupio statističke podatke potrebne planeru za procenu kardinalnosti i troškova potencijalnih planova.

5.2 Eksperiment 1 – Uticaj B-tree indeksa na filtriranje podataka

Prvi eksperiment analizira uticaj indeksa na jednostavan, ali veoma selektivan upit. Cilj je da se uporedi izvršavanje upita bez indeksa nad kolonom koja se koristi za filtriranje sa izvršavanjem istog upita nakon kreiranja B-tree indeksa.

Analiziran je sledeći upit:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE customer_id = 52731;
```

U početnom stanju nad kolonom *customer_id* nije postojao poseban indeks. PostgreSQL je izabrao paralelno sekvencijalno skeniranje tabele:

```
Gather
-> Parallel Seq Scan on orders
    Filter: (customer_id = 52731)
```

Dobijeni deo plana izvršavanja bio je:

```
Gather  (cost=1000.00..15534.43 rows=11 width=40) (actual
time=6.729..18.543 rows=7.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared read=9325
    -> Parallel Seq Scan on orders  (cost=0.00..14533.33 rows=5 width=40)
(actual time=4.156..13.481 rows=2.33 loops=3)
        Filter: (customer_id = 52731)
        Rows Removed by Filter: 333331
        Buffers: shared read=9325
Planning:
    Buffers: shared hit=36
Planning Time: 0.257 ms
Execution Time: 18.607 ms
```

PostgreSQL je planirao korišćenje dva dodatna radna procesa, tako da je sekvencijalno skeniranje izvršavano kroz ukupno tri procesa. Vrednost *Rows Removed by Filter* iznosi približno 333.330 po izvršavanju operatora, dok je operator izvršen tri puta. To znači da je pregledan i odbačen gotovo kompletan sadržaj tabele od milion redova kako bi bilo pronađeno samo 11 odgovarajućih redova.

Interesantan rezultat predstavlja i procena kardinalnosti. PostgreSQL je na nivou *Gather* operatora procenio da će upit vratiti 11 redova, dok je stvarno pronađeno 7 redova. Statistička procena je bila dobra. Problem nije predstavljao pogrešan izbor zasnovan na lošoj proceni, već nepostojanje efikasnijeg načina pristupa traženim podacima.

Vreme izvršavanja upita bez odgovarajućeg indeksa iznosilo je:

Execution Time: 18.607 ms

U drugom delu eksperimenta nad kolonom *customer_id* kreira se B-tree indeks:

```
CREATE INDEX idx_orders_customer_id
ON orders(customer_id);
ANALYZE orders;
```

Nakon kreiranja indeksa isti upit se ponovo analizira:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE customer_id = 52731;
```

PostgreSQL je ovog puta izabrao kombinaciju *Bitmap Index Scan* i *Bitmap Heap Scan* operatora:

```
Bitmap Heap Scan on orders (cost=4.51..47.51 rows=11 width=40) (actual
time=0.279..0.609 rows=7.00 loops=1)
  Recheck Cond: (customer_id = 52731)
  Heap Blocks: exact=7
  Buffers: shared hit=4 read=9
  -> Bitmap Index Scan on idx_orders_customer_id (cost=0.00..4.51
rows=11 width=0) (actual time=0.092..0.124 rows=7.00 loops=1)
    Index Cond: (customer_id = 52731)
    Index Searches: 1
    Buffers: shared hit=3 read=3
Planning:
  Buffers: shared hit=17 read=1
Planning Time: 0.389 ms
Execution Time: 0.656 ms
(12 rows)
```

Bitmap Index Scan koristi kreirani indeks kako bi pronašao lokacije redova koji zadovoljavaju uslov *customer_id = 52731*. Dobijene lokacije se zatim koriste u *Bitmap Heap Scan* operatoru za pristup odgovarajućim podacima u tabeli.

Za razliku od prvog plana, nije bilo potrebno sekvencijalno pregledati približno milion redova. PostgreSQL je pomoću indeksa pronašao 7 odgovarajućih redova, a *Heap Blocks: exact=7* pokazuje da je za njihovo preuzimanje bilo potrebno pristupiti samo 7 odgovarajućih heap blokova.

Nakon kreiranja indeksa ukupno vreme izvršavanja iznosilo je:

Execution Time: 0.656 ms

5.3 Eksperiment 2 – Jednokolonski i višekolonski indeks

Drugi eksperiment ima za cilj da ispita uticaj višekolonskog (*composite*) indeksa na upite koji istovremeno filtriraju podatke prema više kolona. Za početno merenje korišćen je indeks *idx_orders_customer_id*, kreiran u prethodnom eksperimentu nad kolonom *customer_id*.

Analiziran je sledeći upit:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE customer_id = 52731
      AND created_at >= TIMESTAMP '2025-01-01';
```

PostgreSQL je za izvršavanje upita izabrao kombinaciju *Bitmap Index Scan* i *Bitmap Heap Scan* operatora:

```
Bitmap Heap Scan on orders (cost=4.51..47.54 rows=4 width=40) (actual
time=0.362..0.494 rows=2.00 loops=1)
  Recheck Cond: (customer_id = 52731)
  Filter: (created_at >= '2025-01-01 00:00:00'::timestamp without time
zone)
  Rows Removed by Filter: 5
  Heap Blocks: exact=7
  Buffers: shared hit=10
  -> Bitmap Index Scan on idx_orders_customer_id (cost=0.00..4.51
rows=11 width=0) (actual time=0.161..0.161 rows=7.00 loops=1)
    Index Cond: (customer_id = 52731)
    Index Searches: 1
    Buffers: shared hit=3
Planning:
  Buffers: shared hit=3
Planning Time: 2.050 ms
Execution Time: 0.631 ms
(14 rows)
```

Postojeći indeks sadrži samo kolonu *customer_id*, zbog čega PostgreSQL može indeks koristiti za prvi uslov *customer_id = 52731*.

Bitmap Index Scan je pomoću indeksa pronašao ukupno 7 redova koji pripadaju traženom korisniku. Međutim, indeks ne sadrži kolonu *created_at*, pa uslov *created_at >= '2025-01-01'* nije mogao da bude iskorišćen kao deo uslova indeksnog pretraživanja. Nakon pronalaženja 7 kandidata PostgreSQL je pristupio odgovarajućim heap blokovima i naknadno primenio uslov po datumu kao *Filter*.

Od 7 redova pronađenih pomoću indeksa, 5 nije zadovoljilo uslov po datumu i uklonjeno je, dok su 2 reda predstavljala konačni rezultat upita:

```
rows=2
Rows Removed by Filter: 5
Heap Blocks: exact=7
```

Za izvršavanje upita korišćeno je ukupno 10 blokova koji su se već nalazili u shared buffer cache-u:

```
Buffers: shared hit=10
```

Izmereno vreme izvršavanja iznosilo je:

```
Execution Time: 0.631 ms
```

Nakon početnog merenja kreiran je višekolonski indeks nad kolonama *customer_id* i *created_at*:

```
CREATE INDEX idx_orders_customer_created
ON orders(customer_id, created_at);
ANALYZE orders;
```

Nakon toga je ponovo izvršen isti upit:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE customer_id = 52731
      AND created_at >= TIMESTAMP '2025-01-01';
```

PostgreSQL je ponovo izabrao kombinaciju *Bitmap Index Scan* i *Bitmap Heap Scan* operatora, ali je ovog puta korišćen novokreirani višekolonski indeks:

```
Bitmap Heap Scan on orders (cost=4.47..20.28 rows=4 width=40) (actual
time=0.105..0.106 rows=2.00 loops=1)
  Recheck Cond: ((customer_id = 52731) AND (created_at >= '2025-01-01
00:00:00'::timestamp without time zone))
  Heap Blocks: exact=2
  Buffers: shared hit=2 read=3
  -> Bitmap Index Scan on idx_orders_customer_created (cost=0.00..4.46
rows=4 width=0) (actual time=0.097..0.097 rows=2.00 loops=1)
    Index Cond: ((customer_id = 52731) AND (created_at >= '2025-01-01
00:00:00'::timestamp without time zone))
    Index Searches: 1
    Buffers: shared read=3
Planning:
  Buffers: shared hit=17 read=1
Planning Time: 0.657 ms
Execution Time: 0.119 ms
(12 rows)
```

Najznačajnija razlika u odnosu na prethodni plan vidi se u vrednosti *Index Cond*. Kod indeksa koji je sadržao samo kolonu *customer_id*, PostgreSQL je kroz indeks mogao da primeni samo uslov nad tom kolonom, dok se uslov nad *created_at* izvršavao naknadno kao *Filter*.

Nakon kreiranja višekolonskog indeksa oba uslova se koriste direktno prilikom indeksnog pretraživanja:

```
Index Cond:
((customer_id = 52731)
AND (created_at >= '2025-01-01 00:00:00'))
```

Indeks je zato odmah pronašao samo 2 reda koja zadovoljavaju oba uslova. U prethodnom planu indeks je pronalazio 7 redova, nakon čega je dodatnim filtriranjem bilo potrebno odbaciti još 5 redova.

Promena se vidi i u broju heap blokova kojima je potrebno pristupiti. Sa običnim indeksom PostgreSQL je pristupao 7 heap blokova dok je nakon kreiranja višekolonskog indeksa bilo potrebno pristupiti samo 2.

Smanjena je i upotreba shared buffer-a. Početni upit koristio je:

```
Buffers: shared hit=10
```

dok je nakon primene višekolonskog indeksa zabeleženo:

```
Buffers: shared hit=2 read=3
```

Ukupno vreme izvršavanja smanjeno je sa *0.631 ms* na *0.119 ms*.

5.4 Eksperiment 3 – Uticaj načina pisanja upita na korišćenje indeksa

Treći eksperiment ima za cilj da pokaže da samo postojanje indeksa nije dovoljno da bi ga PostgreSQL efikasno iskoristio. Način na koji je uslov u SQL upitu formulisan može direktno uticati na izbor plana izvršavanja.

Pre izvođenja eksperimenta kreiran je indeks nad kolonom *created_at*:

```
CREATE INDEX idx_orders_created_at
ON orders(created_at);
ANALYZE orders;
```

Zatim je analiziran upit kojim se izdvajaju sve porudžbine nastale određenog datuma:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE DATE(created_at) = DATE '2025-05-10';
```

Iako indeks nad kolonom *created_at* postoji, PostgreSQL je za izvršavanje upita izabrao paralelno sekvencijalno skeniranje:

```
Gather (cost=1000.00..17075.00 rows=5000 width=40) (actual
time=1.088..32.190 rows=600.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=1620 read=7705
  -> Parallel Seq Scan on orders (cost=0.00..15575.00 rows=2083
width=40) (actual time=0.825..25.279 rows=200.00 loops=3)
    Filter: (date(created_at) = '2025-05-10'::date)
    Rows Removed by Filter: 333133
    Buffers: shared hit=1620 read=7705
Planning:
  Buffers: shared hit=23 read=1
Planning Time: 0.898 ms
```



```
Execution Time: 32.373 ms  
(12 rows)
```

Razlog je to što se uslov ne primenjuje direktno nad vrednošću kolone *created_at*, već nad rezultatom funkcije *DATE(created_at)*. Običan B-tree indeks nad *created_at* indeksira originalne vrednosti kolone, a ne rezultat navedene funkcije. Zbog toga planer nije mogao da formira odgovarajući *Index Cond* nad postojećim indeksom.

PostgreSQL je zato morao da pročita praktično celu tabelu i za svaki red izračuna vrednost *DATE(created_at)*, nakon čega je proveravao da li ona odgovara datumu 2025-05-10.

Upit je pronašao ukupno 600 redova. Pošto je *Parallel Seq Scan* izvršen kroz tri procesa, vrednost:

```
Rows Removed by Filter: 333133  
loops=3
```

pokazuje da je približno 999.399 redova odbačeno tokom filtriranja.

Plan izvršavanja pokazao je i značajan broj pristupa buffer blokovima:

```
Buffers: shared hit=1620 read=7705
```

Ukupno vreme izvršavanja iznosilo je:

```
Execution Time: 32.373 ms
```

Nakon početnog merenja upit je preformulisan tako da se uslov primenjuje direktno nad kolonom *created_at*, bez korišćenja funkcije *DATE()*:

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT *  
FROM orders  
WHERE created_at >= TIMESTAMP '2025-05-10 00:00:00'  
      AND created_at < TIMESTAMP '2025-05-11 00:00:00';
```

PostgreSQL je nakon ove izmene izabrao kombinaciju *Bitmap Index Scan* i *Bitmap Heap Scan* operatora:

```
Bitmap Heap Scan on orders (cost=14.53..1903.77 rows=596 width=40) (actual  
time=0.564..5.594 rows=600.00 loops=1)  
  Recheck Cond: ((created_at >= '2025-05-10 00:00:00'::timestamp without
```

```

time zone) AND (created_at < '2025-05-11 00:00:00'::timestamp without time
zone))
  Heap Blocks: exact=585
  Buffers: shared hit=118 read=472
  -> Bitmap Index Scan on idx_orders_created_at (cost=0.00..14.39
rows=596 width=0) (actual time=0.314..0.314 rows=600.00 loops=1)
    Index Cond: ((created_at >= '2025-05-10 00:00:00'::timestamp
without time zone) AND (created_at < '2025-05-11 00:00:00'::timestamp
without time zone))
    Index Searches: 1
    Buffers: shared read=5
  Planning Time: 0.171 ms
  Execution Time: 5.666 ms
(10 rows)

```

Za razliku od prethodnog plana, oba uslova nad datumom sada se pojavljuju u okviru *Index Cond*. To znači da PostgreSQL može da koristi B-tree indeks *idx_orders_created_at* za pronalaženje samo onih redova koji pripadaju traženom vremenskom opsegu.

Indeks je pronašao ukupno 600 odgovarajućih redova, što odgovara broju rezultata dobijenom i u prethodnoj verziji upita. Na taj način potvrđeno je da je semantika upita ostala ista, dok se promenio samo način njegovog izvršavanja.

Umesto sekvencijalnog pregleda praktično svih 1.000.000 redova, PostgreSQL je nakon preformulisanja upita pristupio samo heap blokovima koji sadrže relevantne redove *Heap Blocks: exact=585*

Smanjen je i broj buffer operacija. Kod prvobitnog upita zabeleženo je:

```
Buffers: shared hit=1620 read=7705
```

dok je nakon preformulisanja upita dobijeno:

```
Buffers: shared hit=118 read=472
```

Vreme izvršavanja smanjeno je sa 32.373 ms na 5.666 ms.

5.5 Eksperiment 4 – Optimizacija JOIN operacije

Četvrti eksperiment ima za cilj da pokaže uticaj indeksa nad kolonom koja učestvuje u *JOIN* operaciji. Analiziran je upit koji povezuje tabele *customers*, *orders* i *order_items* i računa vrednost porudžbina za jednog konkretnog korisnika:

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    c.id,
    c.name,
    o.id AS order_id,
    SUM(oi.quantity * oi.price) AS order_value
FROM customers c
JOIN orders o
    ON o.customer_id = c.id
JOIN order_items oi
    ON oi.order_id = o.id
WHERE c.id = 52731
GROUP BY
    c.id,
    c.name,
    o.id;

```

U trenutku početnog merenja nad kolonom *order_items.order_id* nije postojao indeks. PostgreSQL je zato za pronalaženje stavki koje pripadaju porudžbinama izabranog korisnika morao da izvrši paralelno sekvencijalno skeniranje cele tabele *order_items*.

Ključni deo plana izvršavanja bio je:

```

Hash Join
  Hash Cond: (oi.order_id = o.id)

-> Parallel Seq Scan on order_items oi
    rows=666666.67 loops=3

-> Hash
    -> Bitmap Heap Scan on orders o
        Recheck Cond: (customer_id = 52731)

```

Tabela *orders* je zahvaljujući prethodno kreiranom indeksu efikasno filtrirana na samo 7 porudžbina korisnika. Međutim, za njihovo povezivanje sa tabelom *order_items* PostgreSQL nije imao odgovarajući indeks nad *order_id*.

Zbog toga je izvršen *Parallel Seq Scan* nad tabelom *order_items*, koja sadrži 2.000.000 redova. Operator je izvršen kroz tri procesa, pri čemu je svaki obradio približno 666666.67 redova što ukupno odgovara praktično celoj tabeli.

PostgreSQL je zatim formirao malu hash strukturu od 7 odgovarajućih porudžbina i proveravao redove iz *order_items* tabele u odnosu na njihove *order_id* vrednosti.

Najveći deo troška plana upravo potiče od sekvencijalnog čitanja velike tabele:

```
Parallel Seq Scan on order_items  
Buffers: shared hit=48 read=16619
```

Ukupno izvršavanje upita zahtevalo je:

```
Buffers: shared hit=110 read=16622
```

dok je vreme izvršavanja iznosilo:

```
Execution Time: 187.772 ms
```

Konačni rezultat sadržao je 7 grupisanih porudžbina.

Nakon početnog merenja kreiran je indeks nad kolonom *order_items.order_id*:

```
CREATE INDEX idx_order_items_order_id  
ON order_items(order_id);  
ANALYZE order_items;
```

Nakon toga je ponovo izvršen isti upit. PostgreSQL je ovog puta izabrao potpuno drugačiji plan izvršavanja:

```
Nested Loop  
-> Nested Loop  
    -> Index Scan using customers_pkey on customers  
    -> Bitmap Heap Scan on orders  
        -> Bitmap Index Scan on idx_orders_customer_created  
  
-> Index Scan using idx_order_items_order_id on order_items  
    Index Cond: (order_id = o.id)
```

Najznačajnija promena odnosi se na pristup tabeli *order_items*. U početnom planu PostgreSQL je koristio *Parallel Seq Scan* i pregledao praktično svih 2.000.000 redova kako bi pronašao stavke koje pripadaju samo 7 relevantnih porudžbina.

Nakon kreiranja indeksa, planer je procenio da je povoljnije koristiti *Nested Loop* i za svaku pronađenu porudžbinu direktno pristupiti odgovarajućim stavkama preko indeksa:

```
Index Scan using idx_order_items_order_id on order_items oi  
Index Cond: (order_id = o.id)
```

```
rows=2.71 loops=7
```

Operator je izvršen 7 puta, po jednom za svaku pronađenu porudžbinu. Umesto skeniranja cele tabele, PostgreSQL je izvršio 7 ciljnih indeksnih pretraga i pronašao ukupno 19 odgovarajućih stavki. Promena plana je značajno smanjila broj potrebnih pristupa podacima.

Pre kreiranja indeksa ukupna upotreba buffer-a bila je:

```
Buffers: shared hit=110 read=16622
```

dok je nakon kreiranja indeksa iznosila:

```
Buffers: shared hit=25 read=28
```

Vreme izvršavanja smanjeno je sa *187.772 ms* na *0.887 ms*.

5.6 Eksperiment 5 – Selektivnost i izbor plana izvršavanja

Poslednji eksperiment ima za cilj da pokaže da postojanje indeksa ne znači da će PostgreSQL uvek izabrati isti način pristupa podacima. Izbor plana zavisi i od selektivnosti uslova, odnosno od toga koliki deo tabele zadovoljava zadati kriterijum.

Za potrebe eksperimenta kreiran je indeks nad kolonom *status* i prvo je analiziran upit koji izdvaja porudžbine sa statusom REFUNDED:

```
CREATE INDEX idx_orders_status  
ON orders(status);
```

```
ANALYZE orders;
```

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT *  
FROM orders  
WHERE status = 'REFUNDED';
```

U generisanim testnim podacima status REFUNDED predstavlja približno 2% ukupnog broja porudžbina. PostgreSQL je za ovaj upit izabrao kombinaciju *Bitmap Index Scan* i *Bitmap Heap Scan* operatora:

```
Bitmap Heap Scan on orders (cost=226.39..9799.73 rows=19867 width=40)
```

```

(actual time=1.799..36.915 rows=20081.00 loops=1)
  Recheck Cond: ((status)::text = 'REFUNDED'::text)
  Heap Blocks: exact=8192
  Buffers: shared hit=2463 read=5749
  -> Bitmap Index Scan on idx_orders_status (cost=0.00..221.43
rows=19867 width=0) (actual time=0.961..0.962 rows=20081.00 loops=1)
    Index Cond: ((status)::text = 'REFUNDED'::text)
    Index Searches: 1
    Buffers: shared read=20
Planning:
  Buffers: shared hit=19 read=1
Planning Time: 0.270 ms
Execution Time: 37.572 ms
(12 rows)

```

Indeks je pronašao 20.081 red koji zadovoljava uslov. Planer je prethodno procenio da će rezultat sadržati 19.867 redova, što predstavlja veoma preciznu procenu kardinalnosti.

Pošto samo mali deo tabele ima status REFUNDED, PostgreSQL je procenio da je korišćenje indeksa povoljnije od sekvencijalnog skeniranja svih 1.000.000 redova.

Bitmap Index Scan je prvo pronašao lokacije relevantnih redova u indeksu, nakon čega je *Bitmap Heap Scan* pristupio heap blokovima koji ih sadrže. Ukupno je bilo potrebno pristupiti 8.192 heap bloka.

Upotreba buffer-a iznosila je:

```
Buffers: shared hit=2463 read=5749
```

dok je ukupno vreme izvršavanja bilo:

```
Execution Time: 37.572 ms
```

U drugom delu eksperimenta analiziran je upit nad istom kolonom i uz postojanje istog indeksa, ali za vrednost COMPLETED:

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE status = 'COMPLETED';

```

Za razliku od prethodnog upita, PostgreSQL ovog puta nije koristio indeks *idx_orders_status*, već je izabrao sekvencijalno skeniranje cele tabele:

```
Seq Scan on orders (cost=0.00..21825.00 rows=701200 width=40) (actual
time=0.026..115.875 rows=701303.00 loops=1)
  Filter: ((status)::text = 'COMPLETED'::text)
  Rows Removed by Filter: 298697
  Buffers: shared hit=8542 read=783
Planning Time: 0.261 ms
Execution Time: 131.985 ms
(6 rows)
```

Upit je vratio ukupno 701.303 reda od 1.000.000 redova u tabeli, odnosno približno 70% svih porudžbina. PostgreSQL planer je procenio da će uslov vratiti 701.200 redova.

Procena kardinalnosti je zbog toga bila veoma precizna. Na osnovu statističkih podataka PostgreSQL je znao da vrednost COMPLETED predstavlja veoma veliki deo tabele i procenio da indeksni pristup nije najpovoljnija strategija. Umesto pronalaženja velikog broja lokacija preko indeksa i naknadnog pristupa odgovarajućim heap blokovima, planer je izabrao direktno sekvencijalno čitanje tabele *Seq Scan on orders*. Tokom skeniranja pronađeno je 701303 odgovarajućih redova, dok je 298697 redova odbačeno.

Upotreba buffer-a iznosila je:

```
Buffers: shared hit=8542 read=783
```

dok je ukupno vreme izvršavanja bilo:

```
Execution Time: 131.985 ms
```

6. Zaključak

Optimizacija upita predstavlja važan deo efikasnog rada sistema za upravljanje bazama podataka, posebno u sistemima koji rade sa velikim količinama podataka. Sa rastom baze, razlike između odgovarajućeg i neodgovarajućeg plana izvršavanja postaju sve izraženije, zbog čega razumevanje načina na koji sistem pristupa i obrađuje podatke ima značajnu ulogu u održavanju dobrih performansi.

PostgreSQL koristi optimizaciju zasnovanu na proceni troškova i na osnovu dostupnih statistika, strukture baze i karakteristika upita bira plan za koji procenjuje da je najpovoljniji. Zbog toga ne postoji jedna tehnika optimizacije koja predstavlja najbolje rešenje u svim situacijama. Efikasnost određenog pristupa zavisi od količine i distribucije podataka, selektivnosti uslova i načina na koji se baza koristi.

Pravilna optimizacija zato zahteva kombinaciju razumevanja rada sistema i praktične analize. Umesto oslanjanja na pretpostavke, potrebno je analizirati planove izvršavanja i meriti efekte napravljenih promena. Na taj način moguće je identifikovati stvarne uzroke problema sa performansama i primeniti rešenja koja odgovaraju konkretnom slučaju.

Može se zaključiti da optimizacija upita nije jednokratan postupak, već proces koji prati razvoj baze i promene u količini i strukturi podataka. Razumevanje planova izvršavanja i odluka koje donosi optimizator predstavlja osnovu za efikasno upravljanje performansama PostgreSQL baza podataka.

7. Literatura

- [1] PostgreSQL Global Development Group, *PostgreSQL 18 Documentation – Using EXPLAIN*, PostgreSQL, 2026. Dostupno na: <https://www.postgresql.org/docs/18/using-explain.html>
- [2] PostgreSQL Global Development Group, *PostgreSQL 18 Documentation – Planner Statistics*, PostgreSQL, 2026. Dostupno na: <https://www.postgresql.org/docs/18/planner-stats.html>
- [3] PostgreSQL Global Development Group, *PostgreSQL 18 Documentation – Indexes*, PostgreSQL, 2026. Dostupno na: <https://www.postgresql.org/docs/18/indexes.html>
- [4] PostgreSQL Global Development Group, *PostgreSQL 18 Documentation – Performance Tips*, PostgreSQL, 2026. Dostupno na: <https://www.postgresql.org/docs/18/performance-tips.html>
- [5] PostgreSQL Global Development Group, *PostgreSQL 18 Documentation – ANALYZE*, PostgreSQL, 2026. Dostupno na: <https://www.postgresql.org/docs/18/sql-analyze.html>
- [6] A. Silberschatz, H. F. Korth, S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
- [7] H. Garcia-Molina, J. D. Ullman, J. Widom, *Database Systems: The Complete Book*, 2nd ed., Pearson, 2008.